

SD2.0虚拟资产库对接文档

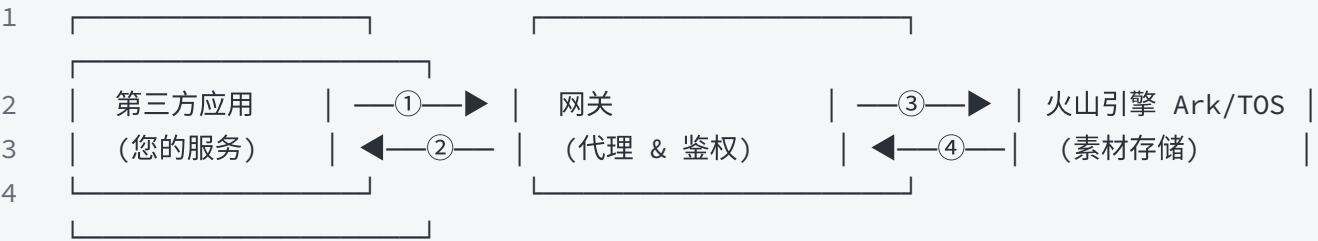
本文档面向需要使用 **Seedance 2.0 人像 / 私域可信素材库** 的第三方接入方。

目录

- [接入架构](#接入架构)
- [前置准备](#前置准备)
- [第一步 获取访问令牌](#第一步-获取访问令牌)
- [第二步 素材组管理](#第二步-素材组管理)
- [第三步 素材操作](#第三步-素材操作)
- [错误码参考](#错误码参考)
- [文件格式与大小限制](#文件格式与大小限制)
- [安全与最佳实践](#安全与最佳实践)

接入架构

代码块



步骤	说明

①	您的服务需要基于 <code>client_id</code> + <code>client_secret</code> 拿到 Bearer token，然后携带 Bearer token 调用网关 API
②	平台返回统一格式的 JSON 响应
③	平台在服务端用自己的火山引擎凭证签名调用 Ark API / TOS
④	火山引擎返回素材数据

前置准备

获取以下凭证：

- 一个应用需要获取一对 `client_id` 和 `client_secret`
- `client_id`：形如 `ndapp_xxxxxxxxxxxxx`
- `client_secret`：一串随机字符串（**仅在创建时显示一次**，请妥善保管）

第一步 获取访问令牌

所有业务接口均需要 Bearer token 认证。token 通过标准 OAuth2 `client_credentials` 模式获取。

请求

代码块

- ```
1 POST /api/v1/oauth/token
2 Content-Type: application/x-www-form-urlencoded
```

| 参数 | 类型 | 必填 | 说明 |
|----|----|----|----|
|----|----|----|----|

|               |        |   |                           |
|---------------|--------|---|---------------------------|
| grant_type    | string | 是 | 固定填<br>client_credentials |
| client_id     | string | 是 | 平台分配的应用 ID                |
| client_secret | string | 是 | 平台分配的应用密钥                 |

## 示例

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/oauth/token \
2 -H "Content-Type: application/x-www-form-urlencoded" \
3 -d
 "grant_type=client_credentials&client_id=ndapp_afda2db16a806a&client_secret=YOU
 R_SECRET"
```

## 响应

代码块

```
1 {
2 "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3 "token_type": "Bearer",
4 "expires_in": 3600
5 }
```

| 字段           | 说明                     |
|--------------|------------------------|
| access_token | JWT 令牌，后续所有请求携带此 token |
| token_type   | 固定为 Bearer             |
| expires_in   |                        |

|  |                          |
|--|--------------------------|
|  | 有效期（秒），默认 3600<br>(1 小时) |
|--|--------------------------|

**注意：**token 过期后需重新获取。建议在过期前几分钟刷新，避免请求中断。

## 通用说明

## 认证方式

所有 `/api/v1/digital-assets/*` 接口统一使用：

代码块

```
1 Authorization: Bearer <access_token>
```

## 可选请求头

| 请求头                             | 说明                           |
|---------------------------------|------------------------------|
| <code>X-External-User-Id</code> | 您系统内的终端用户 ID，写入审计日志，方便追溯操作来源 |

## 响应格式

所有接口返回 JSON，均包含 `success` 字段：

代码块

```
1 {
2 "success": true,
3 "data": { ... }
4 }
```

错误时返回对应 HTTP 状态码和 `detail` 信息：

代码块

```
1 {
2 "detail": "无权访问该素材组"
3 }
```

## 第二步 素材组管理

### 2.1 创建素材组

代码块

```
1 POST /api/v1/digital-assets/groups/create
```

请求体（JSON）：

| 字段 | 类型 | 必填 | 说明 |
|----|----|----|----|
|    |    |    |    |

|             |        |   |                 |
|-------------|--------|---|-----------------|
| name        | string | 是 | 素材组名称，最长 12 字符  |
| description | string | 否 | 素材组描述，最长 100 字符 |

示例：

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/digital-
 assets/groups/create \
2 -H "Authorization: Bearer <access_token>" \
3 -H "Content-Type: application/json" \
4 -d '{
5 "name": "人像素材-正式环境",
6 "description": "Seedance 2.0 正式环境人像素材库"
7 }'
```

响应：

代码块

```
1 {
2 "success": true,
3 "data": {
4 "Id": "grp_abc123def456",
5 "Name": "人像素材-正式环境",
6 "Description": "Seedance 2.0 正式环境人像素材库",
7 "Status": "Active",
8 "AssetCount": 0,
9 "CreateTime": "2026-04-11T10:00:00Z"
10 }
11 }
```

素材组数量受 `max_asset_groups` 配额限制，超限返回 403。

## 2.2 查询素材组列表

代码块

```
1 POST /api/v1/digital-assets/groups
```

请求体（JSON）：

| 字段          | 类型       | 必填 | 默认值          | 说明                            |
|-------------|----------|----|--------------|-------------------------------|
| name        | string   | 否  | ""           | 按名称模糊搜索                       |
| group_ids   | string[] | 否  | null         | 按素材组 ID 过滤                    |
| page_number | int      | 否  | 1            | 页码，从 1 开始                     |
| page_size   | int      | 否  | 20           | 每页条数                          |
| sort_by     | string   | 否  | "CreateTime" | 排序字段：<br>CreateTime /<br>Name |
| sort_order  | string   | 否  | "Desc"       | 排序方向：Asc<br>/ Desc            |

示例：

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/digital-assets/groups \
2 -H "Authorization: Bearer <access_token>" \
3 -H "Content-Type: application/json" \
4 -d '{"page_number": 1, "page_size": 20}'
```

响应：

代码块

```
1 {
2 "success": true,
3 "data": {
4 "Items": [
5 {
6 "Id": "grp_abc123",
7 "Name": "人像素材-正式",
8 "Status": "Active",
9 "AssetCount": 12
10 }
11],
12 "TotalCount": 1,
13 "PageNumber": 1,
14 "PageSize": 20
15 }
16 }
```

只会返回当前应用拥有的素材组，无法查看其他应用的组。

## 2.3 查询素材组详情

代码块

```
1 POST /api/v1/digital-assets/groups/{group_id}
```

路径参数：



|          |        |
|----------|--------|
| 参数       | 说明     |
| group_id | 素材组 ID |

示例：

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/digital-
 assets/groups/grp_abc123 \
2 -H "Authorization: Bearer <access_token>"
```

## 2.4 更新素材组

代码块

```
1 POST /api/v1/digital-assets/groups/{group_id}/update
```

请求体（JSON）：

| 字段          | 类型     | 必填 | 说明  |
|-------------|--------|----|-----|
| name        | string | 否  | 新名称 |
| description | string | 否  | 新描述 |

# 第三步 素材操作

## 3.1 上传素材

代码块

```
1 POST /api/v1/digital-assets/upload
2 Content-Type: multipart/form-data
```

表单字段：

| 字段       | 类型     | 必填 | 说明                      |
|----------|--------|----|-------------------------|
| file     | file   | 是  | 素材文件（图片 / 视频 / 音频）      |
| group_id | string | 是  | 目标素材组 ID                |
| name     | string | 否  | 素材名称，不传则使用文件名（最长 12 字符） |

示例：

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/digital-assets/upload \
2 -H "Authorization: Bearer <access_token>" \
3 -F "file=@portrait.jpg" \
4 -F "group_id=grp_abc123" \
5 -F "name=女性正脸-01"
```

响应：

代码块

```
1 {
2 "success": true,
3 "asset_id": "asset_789xyz",
4 "asset_type": "Image",
5 "status": "Processing",
6 "tos_url": "https://tos-cn-beijing.volces.com/...",
7 "file_name": "portrait.jpg",
8 "file_size": 2048576
9 }
```

上传后素材状态为 `Processing`，处理完成后变为 `Active`。可通过查询素材详情或列表确认状态。

### 3.2 查询素材列表

代码块

```
1 POST /api/v1/digital-assets/list
```

请求体（JSON）：

| 字段                     | 类型                    | 必填 | 默认值               | 说明                                                                  |
|------------------------|-----------------------|----|-------------------|---------------------------------------------------------------------|
| <code>group_ids</code> | <code>string[]</code> | 否  | <code>null</code> | 按素材组 ID 过滤；<br>不传则查询所有有<br>权组                                       |
| <code>statuses</code>  | <code>string[]</code> | 否  | <code>null</code> | 按状态过滤，如<br><code>["Active"]</code> 、<br><code>["Processing"]</code> |
|                        |                       |    |                   |                                                                     |

|             |        |   |              |         |
|-------------|--------|---|--------------|---------|
| name        | string | 否 | ""           | 按名称模糊搜索 |
| page_number | int    | 否 | 1            | 页码      |
| page_size   | int    | 否 | 20           | 每页条数    |
| sort_by     | string | 否 | "CreateTime" | 排序字段    |
| sort_order  | string | 否 | "Desc"       | 排序方向    |

示例：

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/digital-assets/list \
2 -H "Authorization: Bearer <access_token>" \
3 -H "Content-Type: application/json" \
4 -d '{
5 "group_ids": ["grp_abc123"],
6 "statuses": ["Active"],
7 "page_number": 1,
8 "page_size": 20
9 }'
```

响应：

代码块

```
1 {
2 "success": true,
3 "data": {
4 "Items": [
5 {
6 "Id": "asset_001",
7 "GroupId": "grp_abc123",
8 "Name": "女性正脸-01",
9 "Type": "Image",
10 "Status": "Active"
11 }
12],
13 "TotalCount": 1,
```

```
14 "PageNumber": 1,
15 "PageSize": 20
16 }
17 }
```

### 3.3 查询素材详情

代码块

```
1 POST /api/v1/digital-assets/{asset_id}
```

示例：

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/digital-assets/{asset_id}
 \
2 -H "Authorization: Bearer <your_access_token>"
```

响应：

代码块

```
1 {
2 "success": true,
3 "data": {
4 "Id": "asset_789xyz",
5 "GroupId": "grp_abc123",
6 "Name": "女性正脸-01",
7 "Type": "Image",
8 "Status": "Active",
9 "Url": "https://tos-cn-beijing.volces.com/...",
10 "CreateTime": "2026-04-05T14:30:00Z"
```

```
11 }
12 }
```

### 3.4 更新素材名称

代码块

```
1 POST /api/v1/digital-assets/{asset_id}/update
```

请求体（JSON）：

| 字段                | 类型     | 必填 | 说明           |
|-------------------|--------|----|--------------|
| <code>name</code> | string | 是  | 新名称，最长 12 字符 |

示例：

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/digital-
 assets/{asset_id}/update \
2 -H "Authorization: Bearer <your_access_token>" \
3 -H "Content-Type: application/json" \
4 -d '{"name": "测试素材名称更新"}'
```

响应：

代码块

```
1 {"success":true,"data":{"Id":"asset-xxx"}}
```

---

## 3.5 删除素材

代码块

```
1 POST /api/v1/digital-assets/{asset_id}/delete
```

**警告：**删除操作不可恢复。

**示例：**

代码块

```
1 curl -X POST https://platform-api.nodesk.tech/api/v1/digital-
 assets/{asset_id}/delete \
2 -H "Authorization: Bearer <your_access_token>"
```

**响应：**

代码块

```
1 {
2 "success": true
3 }
```

---

## 错误码参考

| HTTP 状态码 | 说明     | 常见场景                                                                |
|----------|--------|---------------------------------------------------------------------|
| 400      | 请求参数错误 | 文件格式不支持、名称为空或超长、 <code>grant_type</code> 错误                         |
| 401      | 认证失败   | token 过期或无效、 <code>client_id</code> / <code>client_secret</code> 错误 |
| 403      | 权限不足   | 访问非本应用拥有的素材组、素材组配额已满                                                |
| 500      | 服务端错误  | 火山引擎 API 调用失败、内部异常                                                  |
| 503      | 服务不可用  | 火山引擎 AK/SK 或 TOS 未配置                                                |

## 文件格式与大小限制

| 类型 | 支持格式           | 最大文件大小 |
|----|----------------|--------|
| 图片 | jpg, jpeg, png | 30 MB  |
| 视频 | mp4, mov       | 50 MB  |
| 音频 | wav, mp3       | 15 MB  |

## 图片尺寸要求

| 约束项    | 允许范围       |
|--------|------------|
| 宽度（px） | 300 ~ 6000 |
|        |            |



|          |            |
|----------|------------|
| 高度（px）   | 300 ~ 6000 |
| 宽高比（宽/高） | 0.4 ~ 2.5  |

不符合尺寸要求的图片将在上传时被拒绝（返回 400 错误及具体原因）。

## 命名长度限制

| 字段                       | 最大字符数 |
|--------------------------|-------|
| 素材组名称（Group Name）        | 12    |
| 素材组描述（Group Description） | 100   |
| 素材名称（Asset Name）         | 12    |

## 程序化查询限制规则

可通过以下接口获取当前平台的所有上传约束，方便在客户端做前置校验：

代码块

```
1 GET /api/v1/digital-assets/constraints
```

### 示例：

代码块

```
1 curl -X GET https://platform-api.nodesk.tech/api/v1/digital-assets/constraints \
2 -H "Authorization: Bearer <access_token>"
```

响应：

代码块

```
1 {
2 "success": true,
3 "data": {
4 "image": {
5 "formats": [".jpeg", ".jpg", ".png"],
6 "max_size_mb": 30,
7 "dimension_range_px": [300, 6000],
8 "aspect_ratio_range": [0.4, 2.5]
9 },
10 "video": {
11 "formats": [".mov", ".mp4"],
12 "max_size_mb": 50
13 },
14 "audio": {
15 "formats": [".mp3", ".wav"],
16 "max_size_mb": 15
17 },
18 "naming": {
19 "group_name_max_length": 12,
20 "group_description_max_length": 100,
21 "asset_name_max_length": 12
22 }
23 }
24 }
```

建议接入方在首次集成时调用此接口缓存限制规则，在客户端上传前做前置校验，避免无效的网络传输。

## 安全与最佳实践

1. **妥善保管** `client_secret`：密钥仅在创建时展示一次。如果遗失，需联系NoDesk AI 管理员轮换密钥（旧密钥立即失效）。
  2. **token 缓存与刷新**： `access_token` 默认有效期 1 小时，建议在本地缓存，过期前提前刷新，避免每次请求都换 token。
  3. **审计追踪**：建议在请求中携带 `X-External-User-Id` 头，方便平台管理员审计操作来源。
  4. **上传后轮询状态**：上传接口返回 `status: "Processing"`，需轮询素材详情接口确认变为 `Active` 后才可用于 Seedance 2.0 生成。
  5. **错误重试**：对 500/503 错误建议指数退避重试；401 错误应先刷新 token 再重试。
- 

## 代码示例

### Python

代码块

```
1 import requests
2
3 BASE_URL = "https://platform-api.nodesk.tech/api"
4
5 # — 1. 获取 token —
6 def get_token(client_id: str, client_secret: str) -> str:
7 resp = requests.post(f"{BASE_URL}/v1/oauth/token", data={
8 "grant_type": "client_credentials",
9 "client_id": client_id,
10 "client_secret": client_secret,
11 })
12 resp.raise_for_status()
13 return resp.json()["access_token"]
14
15 token = get_token("ndapp_xxxx", "your_secret")
16 headers = {"Authorization": f"Bearer {token}"}
17
18 # — 2. 创建素材组 —
19 resp = requests.post(f"{BASE_URL}/v1/digital-assets/groups/create",
20 headers=headers,
21 json={"name": "人像素材-正式", "description": "Seedance 2.0 人像库"})
22 group_id = resp.json()["data"]["Id"]
23
```

```

24 # — 3. 上传素材 —
25 with open("portrait.jpg", "rb") as f:
26 resp = requests.post(f"{BASE_URL}/v1/digital-assets/upload",
27 headers=headers,
28 files={"file": ("portrait.jpg", f, "image/jpeg")},
29 data={"group_id": group_id, "name": "女性正脸-01"})
30 asset_id = resp.json()["asset_id"]
31
32 # — 4. 查询素材状态 —
33 resp = requests.post(f"{BASE_URL}/v1/digital-assets/{asset_id}",
34 headers=headers)
35 print(resp.json()["data"]["Status"]) # Processing → Active
36
37 # — 5. 列出素材 —
38 resp = requests.post(f"{BASE_URL}/v1/digital-assets/list",
39 headers=headers,
40 json={"group_ids": [group_id], "statuses": ["Active"]})
41 print(resp.json()["data"]["Items"])

```

## JavaScript / Node.js

代码块

```

1 const BASE_URL = "https://platform-api.nodesk.tech/api";
2
3 // 1. 获取 token
4 async function getToken(clientId, clientSecret) {
5 const resp = await fetch(`${BASE_URL}/v1/oauth/token`, {
6 method: "POST",
7 headers: { "Content-Type": "application/x-www-form-urlencoded" },
8 body: new URLSearchParams({
9 grant_type: "client_credentials",
10 client_id: clientId,
11 client_secret: clientSecret,
12 }),
13 });
14 const data = await resp.json();
15 return data.access_token;
16 }
17
18 // 2. 调用素材接口
19 async function listGroups(token) {
20 const resp = await fetch(`${BASE_URL}/v1/digital-assets/groups`, {

```

```
21 method: "POST",
22 headers: {
23 Authorization: `Bearer ${token}`,
24 "Content-Type": "application/json",
25 },
26 body: JSON.stringify({ page_number: 1, page_size: 20 }),
27 });
28 return resp.json();
29 }
30
31 // 3. 上传素材
32 async function uploadAsset(token, groupId, file) {
33 const formData = new FormData();
34 formData.append("file", file);
35 formData.append("group_id", groupId);
36 formData.append("name", file.name);
37 const resp = await fetch(`${BASE_URL}/v1/digital-assets/upload`, {
38 method: "POST",
39 headers: { Authorization: `Bearer ${token}` },
40 body: formData,
41 });
42 return resp.json();
43 }
```

## 接口速查表

| 功能       | 方法   | 路径                                              |
|----------|------|-------------------------------------------------|
| 获取 token | POST | /api/v1/oauth/token                             |
| 查询上传限制   | GET  | /api/v1/digital-assets/constraints              |
| 创建素材组    | POST | /api/v1/digital-assets/groups/create            |
| 列出素材组    | POST | /api/v1/digital-assets/groups                   |
| 素材组详情    | POST | /api/v1/digital-assets/groups/{group_id}        |
| 更新素材组    | POST | /api/v1/digital-assets/groups/{group_id}/update |
| 上传素材     | POST | /api/v1/digital-assets/upload                   |
| 列出素材     | POST | /api/v1/digital-assets/list                     |
| 素材详情     | POST | /api/v1/digital-assets/{asset_id}               |
|          |      |                                                 |

|        |      |                                          |
|--------|------|------------------------------------------|
| 更新素材名称 | POST | /api/v1/digital-assets/{asset_id}/update |
| 删除素材   | POST | /api/v1/digital-assets/{asset_id}/delete |

## 本网关接口与火山引擎Ark Asset API 的完整对应关系：

### 素材组 (Asset Groups)

| 本网关接口                                                | 火山引擎 Ark Action  | 说明                       |
|------------------------------------------------------|------------------|--------------------------|
| POST /api/v1/digital-assets/groups                   | ListAssetGroups  | 查询素材组列表，自动过滤为当前 app 拥有的组 |
| POST /api/v1/digital-assets/groups/create            | CreateAssetGroup | 创建素材组，额外增加配额检查 + 归属登记    |
| POST /api/v1/digital-assets/groups/{group_id}        | GetAssetGroup    | 查询素材组详情，增加权限校验           |
| POST /api/v1/digital-assets/groups/{group_id}/update | UpdateAssetGroup | 更新素材组名称/描述，增加权限校验        |

### 素材 (Assets)

| 本网关接口                                         | 火山引擎 Ark Action              | 说明                                        |
|-----------------------------------------------|------------------------------|-------------------------------------------|
| POST /api/v1/digital-assets/list              | ListAssets                   | 查询素材列表，自动限定为有权组                           |
| POST /api/v1/digital-assets/upload            | TOS put_object + CreateAsset | 组合操作：先上传文件到 TOS，再调 Ark 注册素材。小校验，大视频自动分片上传 |
| POST /api/v1/digital-assets/{asset_id}        | GetAsset                     | 查询素材详情，增加权限校验                             |
| POST /api/v1/digital-assets/{asset_id}/update | UpdateAsset                  | 更新素材名称，增加权限校验                             |
| POST /api/v1/digital-assets/{asset_id}/delete | DeleteAsset                  | 删除素材，增加权限校验                               |

## 使用虚拟人像

参考材料：

<https://www.volcengine.com/docs/82379/2291680?lang=zh>

说明：

对写实风格视频，可通过虚拟人像库预置人像来控制角色样貌。每个素材对应一个独立素材 ID (asset ID)，在 **content.<模态>\_url.url** 字段中传入 **asset://<asset ID>** 即可生成视频。

Asset ID 仅用来向模型传入素材，提示词中仍需使用**"素材类型+序号"** 格式引用素材，序号为请求体中该素材在同类素材中的排序。

正确用法：图片1中美妆博主

错误用法：asset-2026\*\*\*\*是美妆博主

示例：

代码块

```
1 import os
2 import time
3 from volcenginesdkarkruntime import Ark
4 client = Ark(
5 # The base URL for model invocation
6 base_url='',
7 api_key=os.environ.get("API_KEY"),
8)
9 if __name__ == "__main__":
10 print("----- create request -----")
11 create_result = client.content_generation.tasks.create(
12 model="", # Replace with Model ID
13 content=[
14 {
15 "type": "text",
16 "text": "固定机位，近景镜头，清新自然风格。在室内自然光下，图片1中美妆
博主面带笑容，向镜头介绍图片2中的面霜。博主将手里的面霜展示给镜头，开心地说“挖到本命面霜
了！”；接着她一边用手指轻轻蘸取面霜展示那种软糯感，一边说“质地像云朵一样软糯，一抹就吸
收”；最后她把面霜涂抹在脸颊上，展示着水润透亮的皮肤，同时自信地说“熬夜急救、补水保湿全搞
定”。要求画面中人物居中，完整展示人物的整个脑袋和上半身，始终对焦人脸，人脸始终清晰，纯净
无任何字幕。"
17 },
18 {
19 "type": "image_url",
20 "image_url": {
21 "url": "asset://asset-20260401123823-6d4x2"
22 },
23 "role": "reference_image"
24 },
25 {
26 "type": "image_url",
27 "image_url": {
28 "url": "https://ark-project.tos-cn-
beijing.volces.com/doc_image/r2v_edit_pic1.jpg"
29 },
30 "role": "reference_image"
31 },
32],
33 generate_audio=True,
34 ratio="adaptive",
```

```
35 duration=11,
36 watermark=True,
37)
38 print(create_result)
39 print("----- polling task status -----")
40 task_id = create_result.id
41 while True:
42 get_result = client.content_generation.tasks.get(task_id=task_id)
43 status = get_result.status
44 if status == "succeeded":
45 print("----- task succeeded -----")
46 print(get_result)
47 break
48 elif status == "failed":
49 print("----- task failed -----")
50 print(f"Error: {get_result.error}")
51 break
52 else:
53 print(f"Current status: {status}, Retrying after 30 seconds...")
54 time.sleep(30)
```